

IFSP - Instituto Federal de Educação, Ciência e Tecnologia de São Paulo
Professor Dr. Gustavo Antonio Furquim

AWS Academy

Aplicação Simples com EC2 e Flask

Parte 1: Iniciando o Laboratório

1. Após realizar login no AWS Academy ir em "Cursos" $\mapsto$ "AWS Academy Learner Lab" no menu lateral esquerdo
2. Dentro do curso ir em "Módulos" $\mapsto$ "Laboratório de aprendizagem da AWS Academy" $\mapsto$ "Iniciar os laboratórios de aprendizagem da AWS Academy"
3. No laboratório, símbolo "AWS ●"
4. Opções do laboratório:
 - "Start Lab": inicia laboratório
 - "End Lab": finaliza laboratório, pode iniciar novamente de onde parou
 - "Reset": reinicia laboratório nas configurações iniciais apagando todo o progresso
5. A qualquer momento podemos selecionar "Start Lab" para reiniciar o tempo do laboratório
6. Selecione a opção "Start Lab" e espere o símbolo da AWS ficar verde ("AWS ●"), isso pode demorar alguns minutos na primeira vez, pois as contas AWS estão sendo criadas

OBS: Leia o "Learner Lab" para mais detalhes

Parte 2: Criação do EC2

1. Selecione "AWS ●", será aberto o "Console Home" da AWS em uma nova janela, faça um tour
2. Dentro do console, selecione $\equiv \mapsto$ "All Services" no canto esquerdo superior perto do nome "AWS" para ver todos os serviços disponíveis
3. Na categoria "Compute" selecione a opção "EC2" (Elastic Compute Cloud)
4. Dentro de "EC2" selecionar "Instances (running)" em "Resources" para verificar as instâncias EC2 em execução
5. Por enquanto não temos nenhuma, então selecione "Launch instances" para criar sua primeira instância EC2
6. Na opção "Name", em **Name and tags**, coloque um nome que desejar para a instância, por exemplo, MyWebServerRNA
7. Para esse tutorial, vamos usar o Debian em "Quick Start" dentro de "Application and OS Images (Amazono Machine Image)"

8. Em "Amazon Machine Image (AMI)" podemos selecionar a imagem que iremos utilizar, verifique que somente as opções com **Free tier eligible** estão disponíveis para o lab
9. Em "Architecture" iremos ficar com a opção "64-bit (x86)" para facilitar o andamento desde laboratório introdutório
10. Perceba que o nome do usuário administrador se encontra em **Username**, essa informação é importante
11. Em **Instance type** selecione uma configuração free tier que desejar, para esse lab iremos utilizar "t3.small" com 2 CPUs e 2GiB de memória
12. **Importante:** em **Key pair (login)** selecione um par de chaves que serão utilizados para conexão via SSH, para esse lab:
 - Em "Key pair name" selecione a chave "vockey"
 - Essa chave deve ser baixada no laboratório da AWS (onde iniciou a seção) em "AWS Details"
 - Salve a chave para uso posterior, **não perca**
13. Nas configurações de rede, **Network settings**, deixar selecionado em "Allow SSH traffic from" a opção "My IP"
14. Selecionar as opções "Allow HTTPS traffic from the internet" e "Allow HTTP traffic from the internet", afinal, será um servidor Web
15. Em **Configure storage** podemos adicionar novas unidades de armazenamento, vamos adicionar uma somente como exemplo
16. Clique em "Add new volume", deixe as opções padrão
17. Finalmente, clique em "Launch instance" no lado direito e espere até a instância EC2 ser criada e iniciar com uma mensagem de **Success**
18. Em "≡ > Instances > Launch an instance" selecione "Instances" para voltar às instâncias, onde deve estar listada a nova instância EC2

Parte 3: Alocando um IP público fixo

Toda vez que uma instância for (re)iniciada, ela irá receber um novo IP público, para evitar isso, vamos criar um "Elastic IP address" e atribuir para nossa instância EC2

1. No menu do canto esquerdo selecione em EC2 > Network & Security > Elastic IPs opção "Allocate Elastic IP address"
2. Deixe as opções padrão e clique em "Allocate"
3. Clique no símbolo de "lápiz" no campo "Name" para criar um nome para o EIP como, por exemplo, "MyWebServerEIP"
4. Salve o IP público descrito em "Allocated IPv4 address"
5. Selecione o EIP na caixa de seleção e clique no menu Actions > Associate Elastic IP Address

6. Em "Instance" selecionar a nossa instância EC2 e clicar em "Associate"
7. Deixe selecionada a caixa em "Allow this Elastic IP address to be reassociated"
8. Clique em "Associate"
9. Volte para as suas instâncias EC2 e verifique se o IP público corresponde ao EIP selecionado em "Instances" > "Details" > "Public IPv4 address"

Parte 4: Conectar via SSH

1. Abra um terminal Linux e vá até a pasta onde salvou a chave
2. Modifique as permissões da chave para que somente o usuário dono possa ler (exemplo: `$ chmod 400 vockey.pem`)
3. Conectar a instância EC2 utilizando o chave, o IP público e o nome do usuário administrador (exemplo: `$ ssh -i vockey.pem admin@18.235.138.74`)
4. Aceite (digite "yes") caso seja questionado sobre se deseja realizar a conexão
5. Outra opção de conexão é:
 - Selecionar a instância EC2 no console da AWS
 - Clicar em "Connect"
 - Selecionar o tab "EC2 Instance Connect"
 - Clicar em "Connect"
 - Um terminal deve abrir
 - Essa opção deve ser utilizada para IPs de fora do Campus, visto que selecionamos a opção "My IP" em "Allow SSH traffic from"

Parte 5: Instalando o Servidor Web

1. Primeiro vamos atualizar nosso sistema (`$ sudo apt update & sudo apt upgrade -y`)
2. Vamos reiniciar *jic* (`$ sudo shutdown -r now`)
3. Conecte na instância novamente e vamos começar a instalação com o seguinte comando:
 - `$ sudo apt install -y apache2 apache2-utils`
4. Se tudo deu certo, você já pode verificar se o servidor web está funcionando abrindo um navegador na sua máquina e digitando o IP público de sua instância EC2, a página padrão do Apache ("It works!") deve ser exibida

Parte 6: Instalando nossa Aplicação Flask

1. Vamos criar uma pasta para nossa aplicação
 - `$ sudo mkdir /var/www/html/myapp`
 - `$ cd /var/www/html/myapp`

- `$ sudo chown admin:admin /var/www/html/myapp`
 - `$ sudo apt install git`
 - `$ git clone https://github.com/gafurquim/simpleFlaskSite.git`
 - `$ mv simpleFlaskSite/* .`
 - `$ rm -rf simpleFlaskSite`
2. Agora, instalando Python
 - `$ sudo apt install python3 python3-pip python3-venv -y`
 3. Criando e ativando o ambiente Python e instalando Flask
 - `$ python3 -m venv venv`
 - `$ source venv/bin/activate`
 - `$ pip3 install flask`
 4. Iniciando a aplicação Flask
 - `$ python3 server.py`
 5. Agora tente acessar a aplicação utilizando a porta listada (Running on `http://127.0.0.1:5000`)
 - Neste caso, utilize no navegador o IP público e a porta listada (por exemplo, `18.235.138.74:500`) e perceba que não funciona
 - Por que não funciona?
 6. Agora, finalize a aplicação Python (`ctrl+c`) e tente:
 - Edite o arquivo `server.py` (`$ pico server.py`), troque a linha `"app.run(debug=True)"` por `"app.run(debug=True, port=80, host="0.0.0.0")"`
 - Pare o Apache, pois ele está usando a porta 80 (`$ sudo systemctl stop apache2.service`)
 - Inicie a aplicação Flask como root (`$ sudo venv/bin/python3 server.py`)
 - Tente agora acessar o servidor (por exemplo, `18.235.138.74` no navegador)

IMPORTANTE: rodar a aplicação como root é uma falha grave de segurança!

Parte 7: Executando a Aplicação com Segurança

1. Criando um usuário seguro para a aplicação e tornando a aplicação dele
 - `$ sudo adduser --system --no-create-home --group gunicorn`
 - `$ pip3 install gunicorn`
 - `$ deactivate`
 - `$ sudo chown -R gunicorn:gunicorn /var/www/html/myapp`

Importante: instalar o gunicorn antes de mudar o dono e grupo dos arquivos
2. De forma insegura, a aplicação ainda pode ser iniciado com:
 - `$ sudo /venv/bin/gunicorn server:app -b 0.0.0.0:80`

Parte 8: Criando um Serviço

1. Vamos criar o arquivo do nosso serviço:

- `$ sudo pico /etc/systemd/system/myapp.service`

2. Preencher com o seguinte conteúdo:

```
[Unit]
Description=My WebApp
After=network.target

[Service]
ExecStart=/var/www/html/myapp/venv/bin/gunicorn server:app -w 4 -b 127.0.0.1:8000
WorkingDirectory=/var/www/html/myapp/
Restart=on-failure
User=gunicorn
Group=gunicorn

[Install]
WantedBy=multi-user.target
```

3. Iniciar o serviço:

- `$ sudo systemctl daemon-reload`
- `$ sudo systemctl enable myapp.service`
- `$ sudo systemctl start myapp.service`
- `$ sudo systemctl status myapp.service`

4. Verifique se o serviço está "active (running)"

Parte 9: Configurando Apache para Redirecionar Chamadas na Porta

1. Nos sites disponíveis vamos reconfigurar

- `$ cd /etc/apache2/sites-available/`
- `$ sudo mv default-ssl.conf default-ssl.conf.back`
- `$ sudo mv 000-default.conf 000-default.conf.back`
- `$ sudo mv *.back ~`
- `$ sudo pico 000-default.conf`
- Substituir o conteúdo de 000-default.conf por:

```
<VirtualHost *:80>
    ProxyPass / http://127.0.0.1:8000/
    ProxyPassReverse / http://127.0.0.1:8000/
</VirtualHost>
```

2. Habilitando e Configurando o Módulo de Proxy

- \$ sudo a2enmod proxy proxy_http
- \$ sudo a2dissite 000-default.conf
- \$ sudo a2ensite 000-default.conf

3. (Re)Iniciando o Apache

- \$ sudo systemctl status apache2.service
- \$ sudo systemctl restart apache2.service

4. Agora, somente testar em um navegador com o IP público do servidor

Bônus 1: Modificar o "My IP" para Acessar via SSH

1. Em "EC2" $\mapsto$ "Instances", selecionar sua instância
2. Abaixo, nas configurações da instância, selecionar "Security"
3. Em **Security Details** encontrar "Security groups" e selecionar o Wizard (algo como "sg-04c5915b6916f3bcb (launch-wizard-1)")
4. Em **Inbound Rules**, temos as regras de requisições aceitas/negadas
5. Clique em "Edit inbound Rules"
6. Na regra SSH, em "Source" seleciona "My IP" e ele será trocado pelo seu IP atual
7. Salve as modificações em "Save rules"

Bônus 2: Configurando um Domínio

1. Vamos mudar a configuração para o domínio "guslabs.net"
2. `cd /etc/apache2/sites-available/`
3. `$ sudo mv 000-default.conf guslabs.net.conf`
4. Mudar o conteúdo do arquivo para:

```
<VirtualHost *:80>
    ServerName guslabs.net
    ServerAlias www.guslabs.net

    ProxyPass / http://127.0.0.1:8000/
    ProxyPassReverse / http://127.0.0.1:8000/
</VirtualHost>
```

5. Reconfigurar o proxy e reiniciar o Apache

- \$ sudo a2enmod proxy proxy_http
- \$ sudo a2dissite 000-default.conf
- \$ sudo a2ensite guslabs.net.conf

- `$ sudo systemctl restart apache2.service`
6. Na conta AWS que é dona do domínio, selecione "AWS" perto de ≡
 7. Utilizando o menu, escolha o serviço "Route 53" em ≡ > All Services > Networking & Content Delivery > Route 53
 8. Em "Route 53", selecione ≡ > Hosted zones
 9. Crie uma zona com o "Domain name:" contendo o nome do seu domínio (por exemplo, guslabs.net)
 10. Clique em "Create hosted zone"
 11. Agora, ainda em "Hosted zones" crie um novo registro em "Create record" com o seguinte conteúdo:
Record type: A - Routes traffic to an IPv4 address and some AWS resources
Value: <MyWebServerEIP> (por exemplo, 18.235.138.74)
 12. Clique em "Create records"
 13. Crie novo registro com o seguinte conteúdo:
Record name: www
Record type: CNAME - Routes traffic to another domain name and some AWS resources
Value: guslabs.net.
 14. Clique me "Create records"
 15. Instalando certificado SSL
 - `$ sudo apt install certbot -y`
 - `$ sudo apt install python3-certbot-apache`
 - `$ sudo certbot --apache`
 - e-mail address
 - y
 - n
 - 1 2
 - `$ sudo crontab -e`
 - 1
 - Adicionar essa linha para renovar automaticamente
`0 4 * * 1 /usr/bin/certbot --renew && /usr/bin/systemctl reload apache2.service`